Architecture-Level Code Translation for Industrial Software Migration

Luhar Ayush*, Dev Trivedi†, Vatsal Patel‡, Vijetha Kamath§, Muhammad Abdul Basit Ur Rahim¶
California State University, Long Beach, CA, USA*†‡§¶
{ayush.luhar01*, devmaharshi.trivedi01†, vatsalmaheshkumar.patel01‡, vijethaupendra.kamath01§}@student.csulb.edu
muhammad.abdulbasiturrahim@csulb.edu¶

***Abstract*—This research focuses on Maven configuration and structure updates and explores how the CodeT5 model can automate the migration of Java 8 projects to Java 17. Manually migrating large Java applications is a tedious and error-prone task that requires significant developer time and attention. To create a high-quality training dataset, Java 8 projects were collected from GitHub and manually converted to Java 17 using large-scale modern language models such as ChatGPT and Claude. Each repository was migrated module-by-module to ensure structural and functional accuracy. This process created a parallel dataset of approximately 100 repositories, formatted as a JSONL file to enable efficient fine-tuning of small CodeT5 models. The model fine-tuning was evaluated using standard metrics, including BLUE, ROUGE, and BERTScore. The results showed a strong semantic agreement between the generated and reference outputs, with a BERT score of 0.982 and a symbolic precision of 99.9%. However, issues related to truncated output and incomplete Java file structure reduced the model's overall performance, with an average output-length ratio of 0.8 and an accuracy of 62% for the Java version. The results demonstrate a precise, accelerated migration of the Java version.**

***Index Terms*—CodeT5, Java Migration, GitHub Repositories, Large Language Models, Transformer Models, Code Automation**

I. Introduction

The transition from Java 8 to Java 17 represents a significant shift in modern enterprise software development. Java 17 provides Long-Term Support (LTS), improved performance, stronger security, and new language features. However, many existing industrial systems continue to operate on older versions such as Java 8 due to migration complexity, legacy dependencies, and cost constraints.

Migrating Java 8 applications—especially those using Maven—is a complicated process. Maven Project Object Model (POM) files define dependencies, plugins, compiler settings, and profiles that must be precisely updated for Java 17 compatibility. Handling these updates manually across large codebases or multiple microservices is tedious and error-prone. As a result, organizations need semi-automated, reliable mechanisms to support version upgrades.

This work investigates whether a fine-tuned CodeT5 model can automate the conversion of Java 8 project structures into Java 17 equivalents. A dataset of parallel Java 8–Java 17 repositories was created using manual migration assisted by LLMs such as ChatGPT and Claude. These aligned pairs were converted to JSONL and used to fine-tune the CodeT5-small model. Evaluation was performed using BLEU, ROUGE, BERTScore, and structural XML accuracy.

The fine-tuned model demonstrated strong semantic consistency and captured higher-level patterns of code transformation. However, limitations arose due to token-length constraints, occasional missing XML elements, and incomplete version upgrades. These findings indicate that CodeT5 can significantly accelerate migration but still requires human validation for production reliability.

II. Related Work

The development of automated code translation has progressed hand in hand with the creation of reliable datasets. Early efforts by Aggarwal et al. [1] were among the first to treat programming languages in the same way as natural languages. Using statistical machine translation to migrate from Python 2 to Python 3, they built a bilingual corpus from open-source projects such as Django and NLTK, demonstrating that even small, aligned datasets can enable meaningful translation.

Building on this foundation, Ahmad et al. [2] introduced the AVATAR dataset—one of the first large-scale, supervised resources for program translation. Containing more than 9,500 parallel Java–Python pairs and 250 embedded unit tests, AVATAR enabled systematic evaluation of neural models such as CodeT5 and PLBART. Its emphasis on functional validation, not just syntactic similarity, exposed a significant gap between surface accuracy and executable correctness.

As data scarcity remained a recurring limitation, Agarwal et al. [3] explored generating pseudo-parallel corpora automatically. Their approach used document similarity measures to align functionally related code snippets from monolingual repositories. Models trained on this synthetic data achieved results comparable to those trained on manually curated corpora, suggesting that translation systems could tolerate imperfect alignment as long as the underlying semantics were preserved.

Xie et al. [4] proposed a data-augmentation pipeline that combined comparable corpora with multiple reference generation. Through automatic unit-test filtering, they maintained only semantically valid examples, improving both diversity and reliability. Huang et al. [5] designed a distillation-based method that converted code into a simplified, language-agnostic intermediate representation, creating scalable pseudo-parallel data capable of training models to generalize across programming paradigms.

Zhu et al. [6] introduced MuST-CoST, a multilingual benchmark covering seven popular languages and focusing on snippet-level translation. Yan et al. [7] presented CodeTransOcean, a 45-language, multi-framework corpus that evaluated not only syntactic accuracy but also runtime executability. Khan et al. [8] advanced this direction with XCodeEval, comprising more than 25 million examples across 11 languages and seven programming tasks. Xue et al. [9] released TransLibEval, a benchmark designed around real-world projects involving third-party libraries such as NumPy, Boost, and TensorFlow.

The methodological landscape of code translation has evolved from rule-based syntax conversion to semantically grounded, contexttranslates them with an LLM, and then uses the red context-aware translation through TranCS, which used simulated execution traces to generate richer representations of source code. Szafraniec et al. [11] integrated compiler-generated Intermediate Representations (IRs), such as LLVM IR, into neural translation, aligning deeper structures like control flow and data dependencies.

Yang et al. [12] introduced UniTrans, which generates test cases from source programs, performs translation with an LLM, and then uses test results to repair errors iteratively. Wang et al. [13] addressed low-resource translation by presenting FSCTrans, which fine-tuned CodeT5 through lightweight prompt modules.

Zhang et al. [14] developed hmCodeTrans, an interactive system that allows developers to guide translations in real time. Yuan et al. [15] proposed TRANSAGENT, a multi-agent translation framework that splits the process into specialized modules. Zhang et al. [16] introduced Oxidizer, a full-scale translation system designed to convert entire projects rather than isolated snippets.

Hong and Ryu [17] designed a C-to-Rust translation pipeline that migrated types while maintaining memory safety. Mahmoud et al. [18] introduced TC-Verifier, a formal system that models translation rules as finite-state machines. Schellhorn et al. [19] pursued a similar goal with VeriCode, which translates formal algebraic specifications directly into C code.

Zhao et al. [20] made an early contribution with G-TransEval, which categorized translation complexity into four levels—token, syntactic, library, and algorithmic. Ahmed et al. [21] developed TRACY, a benchmark focused on runtime and memory efficiency.

Nguyen et al. [22] introduced MBTA (Mutation-Based Translation Assessment), which applied mutation testing to gauge translation robustness. Chen et al. [23] conducted an extensive empirical study, *Lost in Translation*, that categorized 15 common translation bugs. Patel et al. [24] explored the human dimension in *Better Together?*, comparing the productivity and accuracy of humans, AIs, and human–AI teams.

The transition from Java 8 to Java 17 represents one of the most significant changes in modern enterprise software development, with Java 17 introducing Long Term Support (LTS), performance improvements, and enhanced security features.

However, the migration process remains complex, mainly for projects that rely on the Maven build system. The project object model (POM) files used by these systems define dependencies, plugins, and compiler settings that must be carefully updated to match the target Java version. Performing these updates manually can be tedious and error-prone, primarily in organizations supporting hundreds of microservices.

The rapid development of large language models, especially those designed for code understanding and generation such as CodeT5, has made it possible to automate the transformation of structured code. This study validates whether a fine-tuned CodeT5 model can reliably transform Maven POM XML files and associated project configurations from Java 8 to Java 17. The goal of the study is to determine whether this approach can reduce the time and human effort required for migration while maintaining structural and semantic accuracy. This study addresses issues of scalability, accuracy, and automation of code migration.

The goal is for the model to learn the underlying structural and semantic patterns of Java build configurations and determine whether it can reliably reproduce them. It is questionable whether the resulting system can act as a standalone migration assistant or a semi-automated support tool for developers. The importance of this work lies in its contributions to both practical software engineering and academic research on large-scale language models for code conversion.

This model automates parts of the migration process, reducing developer workload, minimizing errors, and accelerating efforts to modernize existing systems. The scientific value lies in demonstrating structured transformations using LLMs and identifying limitations that currently prevent fully autonomous deployment.

This study focuses on projects built with Maven and written in Java 8, intending to create an equivalent to Java 17. Although this study is limited to English documentation and Maven-based configurations, the results provide information applicable to similar structured code migration tasks.

III. Proposed Methodology

This methodology includes data collection, preparation, model selection, fine-tuning and evaluation. A set of open-source Java 8 repos were collected from GitHub to create a foundational dataset. These repositories were selected as they all have Java 8 and multiple modules representing of real-world projects.

After the collection was completed, each repository was manually migrated from Java 8 to Java 17 using State-of-the-Art LLMs, specifically, ChatGPT 5 by OpenAI and Claude Sonnet 4.5 by Anthropic. The method of migration was conducted at both the file and module levels to ensure that each component of the project reflected Java 17 compatibility. Each repository was created solely to perform a sanity check on the migrated code.

Each project was converted into a JSONL file format where every entry represented mapping between a Java 8 to its parallel file written in Java 17. The exact process was repeated for

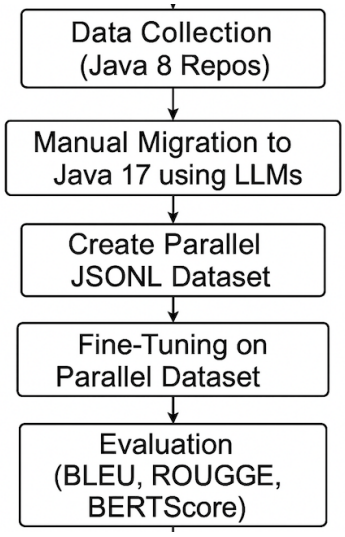

Fig. 1. Proposed Methodology

all 100 repositories, resulting in 100 Java 8 JSONL files and 100 Java 17 JSONL files. These files were then aligned to form a parallel dataset that established a one-to-one correspondence between the legacy and modernized codebases.

Model selection was one of the most essential phases of this research. After experimenting with several models, including CodeBERT and general-purpose T5 variants, Salesforce's CodeT5 was selected for its specialized pretraining on programming languages and its encoder-decoder architecture optimized for both understanding and generating code. Unlike CodeBERT, which is limited to embedding-based understanding and lacks generative prompting capabilities, CodeT5 was explicitly designed for transformation and completion.

Before training could begin, multiple inference experiments were conducted using different prompt templates to assess the model's ability to perform direct migration. Among the various prompts, the best-performing one was selected for the fine-tuning stage. The final optimized prompt used for training and inference was formulated as follows:

"You are an expert Java developer specializing in software version migration. Convert the entire Java project from Java 8 to Java 17, ensuring compatibility with the latest language features, dependency updates, and build configurations. Analyze all modules, classes, and configuration files—including pom.xml, plugins, and dependencies—and generate the fully functional Java 17 version of the project while preserving the original structure and logic."

This fine-tuning was performed on a small CodeT5 model using Hugging Face Transformers and the PyTorch framework. The model was trained in batches of 8 and 10 epochs with a learning rate of 5e-5 using the AdamW optimizer. Due to GPU memory limitations, input and output sequences were limited to a maximum length of 1024 tokens. Training was performed on an NVIDIA A100 GPU with 80 GB of VRAM, which supports mixed precision to improve computational efficiency.

After fine-tuning, the model's performance was evaluated using a combination of quantitative and qualitative measures. BLEU and ROUGE metrics were used to assess n-gram overlap and syntactic accuracy, while BERTScore measured semantic similarity between predicted and reference outputs. Additionally, XML element-level precision, recall, and F1 scores were used to evaluate structural correctness, alongside character-level and token-level analyses to detect truncation or missing content.

IV. Data Preparation

During data collection, open-source Java 8 repositories from GitHub were used to create the base dataset. We then manually converted each repository to Java 17 using modern language models such as ChatGPT and Claude, and carefully tested each module for accuracy and consistency. After conversion, the Java 8 and Java 17 versions of each project compiled and built successfully and were verified to be fully functional. This validation step ensured that the repository did not contain broken dependencies, unresolved plugin configurations, or incompatible library versions before being added to the dataset. Once all projects were verified to build correctly in both environments, the paired versions were organized into a structured parallel corpus and converted to .jsonl format for model training.

The collected dataset contained over 7000 matching modules from approximately 100 GitHub repositories. For each repository, the Java 8 version was matched to the corresponding Java 17 version and manually converted using ChatGPT and Claude. These pairs are checked for structural correctness to ensure that the XML elements are properly formatted and encoded in UTF-8.

The dataset was split into training, validation, and test sets in a 90:10 ratio, with 777 pairs reserved for final evaluation. The average size of Java 8 configuration files was approximately 2,100 characters, while the average size of their Java 17 counterparts was approximately 2,330 characters, an 11% increase due to additional dependencies and configuration options required for compatibility with Java 17. The datasets were carefully validated before being converted to JSONL format for model training to ensure that only well-structured and error-free files were used.

V. Results and Discussion

The fine-tuned CodeT5-small model demonstrated promising yet imperfect results. As shown in Fig. 2, both training and validation losses steadily decreased across epochs, indicating stable convergence and effective learning. The training loss dropped from 0.89 in the first epoch to 0.049 by the tenth epoch, while the validation loss improved from 0.55 to 0.032, confirming that the model generalized well without signs of overfitting.

The model received a BLEU score of 72.26, reflecting high translation quality between the Java 8 and Java 17 representations. As illustrated in Fig. 3, the evaluation metrics demonstrate strong performance across multiple dimensions.

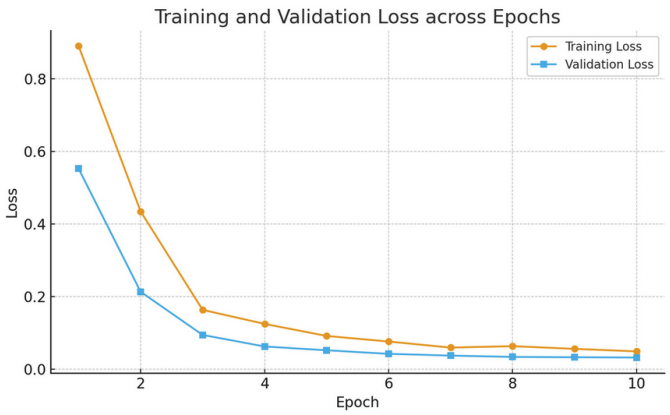

Fig. 2. Training and Validation Loss across Epochs

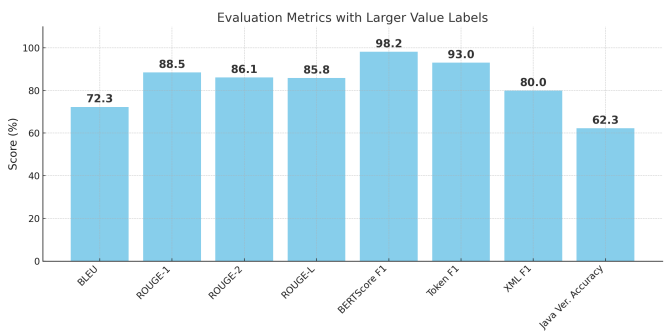

Fig. 3. Evaluation Metrics — BLEU, ROUGE, BERTScore, and XML Accuracy

TABLE I
SUMMARY OF EVALUATION METRICS

Metric	Score
BLEU	72.26
ROUGE-1	0.8848
ROUGE-2	0.8606
ROUGE-L	0.8584
BERTScore F1	0.982
Token Precision	0.9990
Token Recall	0.8821
Token F1	0.9301
Output Length Ratio	0.804
Java Version Accuracy	62.34%
XML Element Recall	69.54%

Table I provides a detailed breakdown of the evaluation metrics. ROUGE-1, ROUGE-2, and ROUGE-L scores of 0.8848, 0.8606, and 0.8584, respectively, further indicate significant n-gram overlap and structural consistency. The BERTScore F1 score of 0.982 indicates good semantic similarity between the generated and reference results, highlighting the model's ability to maintain contextual meaning during transformation. Token-level analysis shows a precision of 0.9990, recall of 0.8821, and F1 score of 0.9301, suggesting that the model rarely generates invalid tokens.

However, some gaps remained. The average output length ratio is 0.804, indicating that the results are approximately 20% shorter than the baseline output, a clear sign of truncation due to the 512 token sequence limit. The accuracy of Java version conversion was 62.34%. This means that approximately one-third of migrated files failed to update their version specifications or dependencies correctly. The 69.54% recall for the XML element also indicates missing secondary dependencies and incomplete plugin configuration.

Qualitatively, the model reflects high-level structure and semantics, but may omit less visible XML components. The following figures provide real examples of the Java 8 input and the corresponding Java 17 output generated by the model, as well as a comparison of the XML configurations before and after migration.

Fig. 4. Example of Java 8 Source Code Used as Model Input

Compared to manual migration, which takes 5–10 minutes per file but yields near-perfect accuracy, this model requires human review but can perform the same task in less than a second.

VI. LIMITATIONS

Although the CodeT5-small model performed well in semantic understanding and structural reproduction, several limitations were identified:

- **Token-length limitations:** The 1024-token cap caused truncated outputs, particularly for large Maven POM files.
- **Missing XML components:** Some secondary dependencies, plugin configurations, and nested tags were occasionally omitted.

```java
public class OrderService {

    public List<String> getFormattedOrders() {
        var orders = new ArrayList<String>();
        orders.add("Order-3");
        orders.add("Order-1");
        orders.add("Order-2");

        orders.sort((a, b) -> a.compareTo(b));  ← Java 17: Lambda sorting

        var now = LocalDateTime.now();  ← Java 17: Modern date/time API
        var formatter = DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss");

        orders.add("Generated at: " + formatter.format(now));

        Object msg = "Order list finalized";
        if (msg instanceof String s) {
            orders.add(s);
        }

        return orders;
    }
}
```

Fig. 5. Generated Java 17 Output Code from the Model

```xml
    <properties>
        <maven.compiler.source>1.8</maven.compiler.source>
        <maven.compiler.target>1.8</maven.compiler.target>
    </properties>

    <dependencies>
        <!-- Old versions -->
        <dependency>
            <groupId>junit</groupId>
            <artifactId>junit</artifactId>
            <version>4.12</version>
            <scope>test</scope>
        </dependency>
    </dependencies>

</project>
```

Fig. 6. Java 8 pom.xml Configuration Before Migration

- **Version accuracy:** Only 62.34% of transformed files correctly updated Java version attributes and configuration values.
- **Dataset constraints:** Only Maven-based Java projects were included, limiting generalizability.
- **Human review requirement:** Outputs often needed developer inspection to ensure functional correctness.

These limitations constrain CodeT5's standalone deployment for enterprise migrations.

```xml
    <properties>
        <maven.compiler.source>17</maven.compiler.source>
        <maven.compiler.target>17</maven.compiler.target>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    </properties>

    <dependencies>
        <!-- Modern JUnit -->
        <dependency>
            <groupId>org.junit.jupiter</groupId>
            <artifactId>junit-jupiter</artifactId>
            <version>5.10.0</version>
            <scope>test</scope>
        </dependency>
    </dependencies>
```

Fig. 7. Java 17 pom.xml Configuration After Migration

VII. Implications of Research

This research provides several broader implications:

- **Scalable migration workflows:** LLM-assisted automation can reduce developer workload across hundreds of microservices.
- **Cross-language applicability:** The approach may generalize to Python 2→3, C→Rust, or framework-level migrations.
- **Productivity enhancements:** Developers can rely on LLM-generated drafts rather than performing full manual rewrites.
- **Foundation for multi-agent systems:** Future migration pipelines may involve dedicated agents for dependency analysis, validation, and refactoring.

These implications highlight the potential of LLMs for large-scale software modernization.

VIII. Conclusions and Future Works

The study demonstrates that our team successfully designed, constructed, and evaluated an end-to-end workflow for automated Java migration using a fine-tuned CodeT5-small model. We collected and manually migrated over one hundred Java 8 repositories, created a large parallel dataset, developed optimized prompt templates, and fine-tuned the model to learn the structural and semantic transformations required to produce Java 17–compatible configurations. Our results show that the model can reliably capture the underlying patterns of version migration and generate meaningful, structurally coherent outputs. However, challenges such as truncated results, occasional missing dependencies, and inconsistent version updates indicate that the model cannot function as a fully autonomous migration tool yet. These limitations are largely due to the compact model size and sequence length restrictions, which constrain its ability to handle complex or lengthy project files.

Overall, this work demonstrates that fine-tuned language models like CodeT5 can significantly reduce manual workload during Java migration.

Acknowledgment

The authors would like to thank California State University, Long Beach, for providing the resources and support necessary for this research.

References

[1] K. Aggarwal et al., "Using Machine Translation for Converting Python 2 to Python 3 Code," *PeerJ Preprints*, 2015.
[2] W. Ahmad et al., "AVATAR: A Parallel Corpus for Java–Python Program Translation," 2023.
[3] M. Agarwal et al., "Using Document Similarity Methods to Create Parallel Datasets for Code Translation," arXiv:2110.05423, 2021.
[4] Y. Xie et al., "Data Augmentation for Code Translation with Comparable Corpora and Multiple References," arXiv, 2023.
[5] Y. Huang et al., "Program Translation via Code Distillation," Microsoft Research, 2023.
[6] M. Zhu et al., "Multilingual Code Snippets Training for Program Translation (MuST-CoST)," *AAAI*, 2022.
[7] W. Yan et al., "CodeTransOcean: A Comprehensive Multilingual Benchmark for Code Translation," arXiv, 2023.

[8] M. Khan et al., "XCodeEval: An Execution-Based Large-Scale Multilingual Multitask Benchmark for Code Understanding, Generation, Translation and Retrieval," *ICLR*, 2024.
[9] P. Xue et al., "A New Benchmark for Evaluating Code Translation with Third-Party Libraries (TransLibEval)," *ACM*, 2025.
[10] W. Sun et al., "Code Search based on Context-Aware Code Translation," *ICSE*, 2022.
[11] M. Szafraniec et al., "Code Translation with Compiler Representations," Meta AI, 2023.
[12] Z. Yang et al., "Exploring and Unleashing the Power of Large Language Models in Automated Code Translation," *ACM FSE*, 2024.
[13] Y. Wang et al., "Few-Shot Code Translation via Task-Adapted Prompt Learning," 2023.
[14] H. Zhang et al., "hmCodeTrans: Human–Machine Interactive Code Translation," *IEEE*, 2024.
[15] Z. Yuan et al., "Semantic Alignment-Enhanced Code Translation via an LLM-Based Multi-Agent System," Fudan University, 2025.
[16] H. Zhang et al., "Scalable, Validated Code Translation of Entire Projects using Large Language Models," University of Bristol & Amazon, 2024.
[17] J. Hong and S. Ryu, "Type-Migrating C-to-Rust Translation using a Large Language Model," *Empirical Software Engineering*, 2024.
[18] A. T. Mahmoud et al., "TC-Verifier: Trans-Compiler-Based Code Translator Verifier with Model Checking," *Applied System Innovation*, 2025.
[19] G. Schellhorn et al., "VeriCode: Correct Translation of Abstract Specifications to C Code," *IFM*, 2025.
[20] L. Zhao et al., "On the Evaluation of Neural Code Translation: Taxonomy and Benchmark (G-TransEval)," Stanford University, 2022.
[21] R. Ahmed et al., "TRACY: Benchmarking Execution Efficiency of LLM-Based Code Translation," University of Illinois, 2023.
[22] P. Nguyen et al., "Mutation Analysis for Evaluating Code Translation (MBTA)," National University of Singapore, 2023.
[23] Y. Chen et al., "Lost in Translation: A Study of Bugs Introduced by LLMs," Microsoft Research, 2024.
[24] D. Patel et al., "Better Together? An Evaluation of AI-Supported Code Translation," Carnegie Mellon University, 2024.